

SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING	
ProgramName: B. Tech		Assignment Type: Lab	AcademicYear: 2025-2026
CourseCoordinatorName		Venkataramana Veeramsetty	
Instructor(s)Name		Dr. V. Venkataramana (Co-ordinator)	
		Dr. T. Sampath Kumar	
		Dr. Pramoda Patro	
		Dr. Brij Kishor Tiwari	
		Dr.J.Ravichander	
		Dr. Mohammand Ali Shaik	
		Dr. Anirodh Kumar	
		Mr. S.Naresh Kumar	
		Dr. RAJESH VELPULA	
		Mr. Kundhan Kumar	
		Ms. Ch.Rajitha	
		Mr. M Prakash	
		Mr. B.Raju	
		Intern 1 (Dharma teja)	
		Intern 2 (Sai Prasad)	
		Intern 3 (Sowmya)	
NS_2 (Mounika)			
CourseCode	24CS002PC215	CourseTitle	AI Assisted Coding
Year/Sem	II/I	Regulation	R24
Date and Day of Assignment	Week3 - Thursday	Time(s)	
Duration	2 Hours	Applicable to Batches	
AssignmentNumber: 6.4(Present assignment number)/24(Total number of assignments)			
Name: V.CHANDANA Hallticket no : 2403a51186 Batch no : 09			
Q.No.	Question		Expected Time to complete
1	Lab 6: AI-Based Code Completion – Classes, Loops, and Conditionals Lab Objectives:		Week3 - Thursday

- To explore AI-powered auto-completion features for core Python constructs.
- To analyze how AI suggests logic for class definitions, loops, and conditionals.
- To evaluate the completeness and correctness of code generated by AI assistants.

Lab Outcomes (LOs):

After completing this lab, students will be able to:

- Use AI tools to generate and complete class definitions and methods.
- Understand and assess AI-suggested loops for iterative tasks.
- Generate conditional statements through prompt-driven suggestions.
- Critically evaluate AI-assisted code for correctness and clarity.

Task Description #1:

- Start a Python class named Student with attributes name, roll_number, and marks. Prompt GitHub Copilot to complete methods for displaying details and checking if marks are above average.

```

class Student:
    def __init__(self, rollno, name, marks):
        self.rollno = rollno
        self.name = name
        self.marks = marks

    def display_student(self):
        print(f"Roll Number: {self.rollno}, Name: {self.name}, Marks: {self.marks}")

    def has_passed(self):
        return self.marks > 40

# Test cases
s1 = Student(101, "Aisha", 85)
s1.display_student()
print(f"Has passed: {s1.has_passed()}")

s2 = Student(102, "Ravi", 39)
s2.display_student()
print(f"Has passed: {s2.has_passed()}")

```

Roll Number: 101, Name: Aisha, Marks: 85
Has passed: True
Roll Number: 102, Name: Ravi, Marks: 39
Has passed: False

Expected Outcome #1:

- Completed class with Copilot-generated methods like display_details() and is_passed(), demonstrating use of if-else conditions.

Task Description #2:

- Write the first two lines of a for loop to iterate through a list of numbers. Use a comment prompt to let Copilot suggest how to calculate and print the square of even numbers only.

```

6] numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9]
    even_squares = [num**2 for num in numbers if num % 2 == 0]
    print(even_squares)

```

[4, 16, 36, 64]

Expected Outcome #2:

- A complete loop generated by Copilot with conditional logic (if number % 2 == 0) and appropriate output.

Task Description #3:

- Create a class called BankAccount with attributes account_holder and balance. Use Copilot to complete methods for deposit(), withdraw(), and check for insufficient balance.

```

class BankAccount:
    def __init__(self, account_holder, balance=0):
        self.account_holder = account_holder
        self.balance = balance
    def deposit(self, amount):
        if amount > 0:
            self.balance += amount
            print(f"Deposited {amount}. New balance is {self.balance}")
        else:
            print("Deposit amount must be positive.")
    def withdraw(self, amount):
        if amount > 0:
            if self.balance >= amount:
                self.balance -= amount
                print(f"Withdrew {amount}. New balance is {self.balance}")
            else:
                print("Insufficient balance.")
        else:
            print("Withdrawal amount must be positive.")
    def check_balance(self):
        print(f"Current balance: {self.balance}")

```

```

➡ Current balance: 0
Deposited 100. New balance is 100
Withdrew 50. New balance is 50
Insufficient balance.
Current balance: 50

```

Expected Outcome #3:

- Functional class with complete method definitions using if conditions and self attributes. Code should prevent overdrawing.

Task Description #4:

- Define a list of student dictionaries with keys name and score. Ask Copilot to write a while loop to print the names of students who scored more than 75.

```

students = [
    {"name": "Alice", "score": 85},
    {"name": "Bob", "score": 70},
    {"name": "Charlie", "score": 92},
    {"name": "David", "score": 65},
    {"name": "Eve", "score": 78}
]

i = 0
while i < len(students):
    if students[i]["score"] > 75:
        print(students[i]["name"])
    i += 1

```

```

→ Alice
  Charlie
  Eve

```

Expected Outcome #4:

- A complete while loop generated by Copilot with proper condition checks and formatted output.

Task Description #5:

- Begin writing a class ShoppingCart with an empty items list. Prompt Copilot to generate methods to add_item, remove_item, and use a loop to calculate the total bill using conditional discounts.

```

class ShoppingCart:
    def __init__(self):
        self.items = []

    def add_item(self, item, price, quantity=1):
        # Check if item already exists in the cart
        for cart_item in self.items:
            if cart_item['item'] == item:
                cart_item['quantity'] += quantity
                print(f"Added {quantity} more of {item}. Total quantity: {cart_item['quantity']}")
                return

        # If item does not exist, add it to the cart
        self.items.append({'item': item, 'price': price, 'quantity': quantity})
        print(f"Added {quantity} x {item} to the cart.")

    def remove_item(self, item, quantity=1):
        for cart_item in self.items:
            if cart_item['item'] == item:
                if cart_item['quantity'] > quantity:
                    cart_item['quantity'] -= quantity
                    print(f"Removed {quantity} of {item}. Remaining quantity: {cart_item['quantity']}")
                elif cart_item['quantity'] == quantity:
                    self.items.remove(cart_item)
                    print(f"Removed all {quantity} of {item} from the cart.")
                else:
                    print(f"Cannot remove {quantity} of {item}. Only {cart_item['quantity']} in the cart.")

```

```

        print(f"Cannot remove {quantity} of {item}. Only {cart_item['quantity']} in the cart.")
        return
    print(f"{item} not found in the cart.")

    def calculate_total(self):
        total = 0
        for item in self.items:
            total += item['price'] * item['quantity']

        # Apply conditional discount (example: 10% off if total is over 100)
        if total > 100:
            discount = total * 0.10
            total -= discount
            print(f"Applied 10% discount. Discount amount: {discount:.2f}")

        return total

# Example Usage
cart = ShoppingCart()
cart.add_item("Laptop", 1200, 1)
cart.add_item("Mouse", 25, 2)
cart.add_item("Keyboard", 75, 1)
print(f"Cart total: ${cart.calculate_total():.2f}")

cart.remove_item("Mouse", 1)
print(f"Cart total after removing a mouse: ${cart.calculate_total():.2f}")

```

```

print(f"Cart total: ${cart.calculate_total():.2f}")

cart.remove_item("Mouse", 1)
print(f"Cart total after removing a mouse: ${cart.calculate_total():.2f}")

cart.remove_item("Laptop", 1)
print(f"Cart total after removing laptop: ${cart.calculate_total():.2f}")

cart.remove_item("Pen", 1) # Example of removing an item not in the cart

```

 Added 1 x Laptop to the cart.
 Added 2 x Mouse to the cart.
 Added 1 x Keyboard to the cart.
 Applied 10% discount. Discount amount: 132.50
 Cart total: \$1192.50
 Removed 1 of Mouse. Remaining quantity: 1
 Applied 10% discount. Discount amount: 130.00
 Cart total after removing a mouse: \$1170.00
 Removed all 1 of Laptop from the cart.
 Cart total after removing laptop: \$100.00
 Pen not found in the cart.

Expected Outcome #5:

- A fully implemented ShoppingCart class with Copilot-generated loops and if-else statements handling item management and discount logic.

Note: Report should be submitted a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots

Evaluation Criteria:

Criteria	Max Marks
Class	1
Loop	1
condition	0.5
Total	2.5 Marks

